Inteligență Artificială: Tema 1 - Introducere în Machine Learning

Seria CA

Bogdan Sprîncenatu, Alexandru-Victor Baciu, Cătălin-Mihail Chiru,
Ana-Maria Simion, Andrei Dugăeșescu, Andrei Olaru

1. Descriere generală

În practica de zi cu zi a unui inginer sau cercetător în domeniul inteligenței artificiale, și al învățării automate în particular, intră frecvent următoarele trei aspecte:

- Vizualizarea și "explorarea" datelor unei probleme (eng. Exploratory Data Analysis - EDA)
- Încercarea de a observa corelații și de a extrage atribute din date pentru a facilita predicția obiectivului ales (e.g. clasificare, regresie, detectie de anomalii)
- Evaluarea mai multor modele pentru găsirea soluției celei mai bune pentru problema dată

Sarcinile de lucru solicită utilizarea de biblioteci de **vizualizare a datelor (crearea de diagrame)**, **extragerea de atribute (feature extraction)** pentru folosirea algoritmilor de clasificare și regresie discutați la curs (folosind oricare dintre implementările voastre de laborator, sau cele dintr-o bibliotecă existentă, precum **scikit-learn**).

Scopul acestei teme constă în familiarizarea cu conceptele de învățare automată (Arbori de decizie, Regresie Liniară, Regresie Logistică), abordarea unei probleme de inteligență artificială, jurnalizarea seturilor de experimente, precum și analiza și susținerea observațiilor dobândite pe parcursul antrenării modelelor.

2. Descrierea Seturilor de Date

Se propune următorul set de date, având ca obiectiv **clasificarea și regresia**.

Scopul este să antrenați modele de învățare care să reușească să prezică valoarea variabilelor țintă cu performanțe cât mai bune. Găsiți setul de date corespunzător seriei la care sunteți înrolați pe metacursul de pe Moodle la secțiunea aferentă acestei teme.

Fiecare set de date este furnizat sub forma unui fișier CSV, având pe prima linie numele coloanelor.
Fiecare set de date are o împărțire din oficiu a datelor în:

- Set de date pentru **antrenare** (sufixul **_train**)
- Set de date pentru **validare** (sufixul **_val**)
- Set de date pentru **testare (etichetele vă sunt ascunse)**

! Atenție - pentru seturile de antrenare și validare veți primii totul ca fișiere csv. Va fi sarcina voastră să izolați obiectivele $y_{clasificare}$, $y_{regresie}$, deoarece în setul de testare, csv-ul nu va conține coloanele cu obiective.

Predicția notelor și a calificativelor - Open University Learning Analytics Dataset (OUALD)

Setul de date aparține platformei de învățare online Open University pe care studenții din afara campusului o utilizează pentru accesarea conținutului cursului, discuții pe forum, trimiterea și verificarea notelor la teme. Acesta constă din 7 cursuri selectate (menționate ca module în setul de date). Diferitele prezentări sunt indicate cu literele „B” și „J” după an pentru semestrul 2, respectiv semestrul 1.

Obiectivele predicției din setul de date:

- Clasificare Multiclasă - final_result (Withdrawn, Fail, Pass, Distinction)
- Regresie - final_coursework_score

Atributele pentru fiecare student sunt următoarele:

Nume atribut	Tip de date	Semnificație
code_module	categorial	Codul de identificare al modulului căruia îi aparține notarea.
code_presentation	categorial	Codul de identificare al prezentării căruia îi aparține notarea.
id_student	numeric	Codul unic de identificare a fiecărui student
gender	categorial / binar	Sexul studentului
region	categorial	Identifică regiunea geografică în care a locuit studentul în timpul prezentării modulului.
highest_education	categorial	Cel mai înalt nivel de educație al studentului la intrarea în modul.
imd_band	categorial	Măsură a calității internetului locului în care a locuit studentul în timpul prezentării modulului.
age_band	categorial	Intervalul de vârstă în care se încadrează studentul
num_of_prev_attempts	numeric	Numărul de încercări anterioare de promovare a modulului.
studied_credits	numeric	Numărul total de credite pentru modulele studiate în prezent.
disability	categorial / binar	Indică dacă elevul a declarat că are o dizabilitate.
…		
final_result	categorial	Rezultatul final al studentului la prezentarea modulului
final_coursework_score	numeric	Nota obținută de student la curs

Descrierea completă a setului de date este disponibilă la OUALD.

3. Cerințe

3.1. Explorarea Datelor (Exploratory Data Analysis) [2p]

Primul pas recomandat în rezolvarea unei probleme de inteligență artificială este obținerea unor informații asupra caracteristicilor principale ale problemei. De regulă, foarte folositoare în această etapă este aplicarea unor metode de ***vizualizare a datelor*** și de ***raportare a distribuțiilor de valori*** pe fiecare variabila, coroborată cu obiectivul învățării.

Exemple de EDA de pe platforma kaggle:

- Pentru **Clasificare** - Titanic EDA & Classification End to End
- Pentru **Regresie** - House Prices EDA.

Este, de asemenea, esențial a se identifica dacă:

- Există atribute cu **valori lipsă**
- Există atribute cu **valori extreme** (eng. outlier identification)
- Există atribute **redundante / neinformative**

! NU PIERDEȚI FOARTE MULT TIMP CU ACEASTĂ ETAPĂ DOAR DE DRAGUL DE A O PUNE ÎN PREZENTARE, EA VA VENI NATURAL PE MĂSURĂ CE ANTRENAȚI ȘI OBSERVAȚI LIMITĂRILE MODELELOR!

RECOMANDARE: Încercați să vă încadrați în **MAXIM** 15 grafice de analiză EDA, pentru a nu avea o documentație extrem de stufoasă, concentrați-vă pe partea de antrenat modele.

Analize cerute

1. Analiza tipului de atribute și a plajei de valori a acestora

Înainte de utilizarea unui model de ML pentru un set de date este importantă identificarea tipurilor de atribute (features) din setul de date și a valorilor acestora.

Este relevantă distincția dintre:

- Atribute cu **valori numerice continue**
- Atribute cu **valori discrete / categoriale** (e.g. education_level/ highest_education - Bachelor, PhD, High School, Diploma, Master / HE Qualification, A Level or Equivalent, Lower Than A Level, gender - M, F)
    - **Observație -** conversia acestor valori în spațiul numeric (e.g. Bachelor - 0, PhD - 1, etc) produce de la sine semnificații semantice diferite - de exemplu Bachelor < PhD < High School. De aceea trebuie să avem grijă cu ordinea lor. Există situații în care nu există o asociere optimă și atunci trebuie să luăm decizii individuale cu felul în care tratăm acele atribute - le eliminăm, suportăm schimbarea structurii

spațiului în care acestea se găsesc și avem grijă să ținem în modele noastre seama de acest aspect.

- Atribute **ordinale** - unde valorile indică o relație de ordine (e.g. număr de steluțe la recenzia unui produs). Atributele ordinale au de regulă o valoare numerică, dar valoarea este subiectivă (e.g. Între un produs *cu rating* 3 și unul cu rating 5, diferența nu este doar valoarea numerică 2).

Pentru fiecare tip de atribut este relevantă extragerea unor statistici la nivelul exemplelor din setul de date.

Pentru atribute numerice continue extrageți și **prezentați într-un tabel**

- numărul de exemple din setul de date care **nu** au valori lipsă
- valoarea medie
- deviația standard a valorilor
- valoarea minimă
- valoarea percentilei de 25% (valoarea sub care se găsesc 25% din exemple)
- valoarea percentilei de 50% (valoarea sub care se găsesc 50% din exemple) - aceasta este valoarea mediană
- valoarea percentilei de 75% (valoarea sub care se găsesc 75% din exemple)
- valoarea maximă

Afișati grafice de tip Boxplot (a se vedea indicațiile din Anexă: Explorarea datelor) prin care să observați vizual plajele de valori ale atributelor numerice continue.

Pentru atribute discrete sau ordinale extrageți și prezentați într-un tabel:

- Numărul de exemple din setul de date care **nu** au valori lipsă
- Numărul de **valori unice**

Prezentați grafice de tip histogramă (a se vedea indicațiile din Anexă: Explorarea datelor) pentru a observa vizual care este distribuția valorilor pentru fiecare atribut categorial / ordinal peste exemplele din setul de date.

CERINȚA MINIMĂ pentru obținerea punctajului maxim pentru această cerință constă în analiza a **MINIM 3** atribute distincte.

2. Analiza echilibrului de clase

Realizați un grafic al frecvenței de apariție a fiecărei etichete (clase) în setul de date de antrenare / validare, folosind **bar plot** / **count plot** (a se vedea indicațiile din Anexă: Explorarea Datelor).
Pentru realizarea unor astfel de bar plots puteți folosi mai multe biblioteci:

- Folosind biblioteca seaborn pentru barplot sau countplot
- Direct dintr-un DataFrame Pandas folosind pandas.DataFrame.plot.bar

Dacă într-un set de date numărul de exemple aferent unui subset de clase este semnificativ mai mare decât exemplele aferente celorlalte clase, atunci clasificarea va fi mai dificilă, riscându-se predicții eronate pentru clasele cu *suport* (i.e. număr de exemple etichetate cu clasa respectivă) scăzut.
În cazul unui set de date cu suport al clase dezechilibrat se va insista pe raportarea metricilor de performanță de tip **precizie, recall** și **F1**. Deoarece, în antrenarea modelului acestea nu pot produce o cale de rezistență mică precum se întâmplă cu acuratețea (Dacă avem un set de date disproporționat, un model care prezice doar clasa majoritară va avea acurateță foarte mare - spre 90%, făcând metrica irelevantă.)

3. Analiza corelației între atribute

1. **Realizați analize** de **corelație** (a se vedea indicațiile din Anexă: Explorarea Datelor) între **atributele numerice continue**, pentru a identifica dacă există atribute redundante.

2. **Realizați analize** de **corelație** între **atributele categorice,** pentru a identifica dacă existe atribute redundante.

Două atribute care sunt puternic corelate (valori ale indicelui de corelație apropiate de -1 sau 1) vor avea, în mare măsură, aceeași putere de predicție, astfel încât utilizarea amândurora într-un algoritm de predicție nu aduce performanțe suplimentare, ci doar adaugă la complexitatea de calcul.

Ca atare o analiză a corelației pe perechi de atribute ne poate indica dacă putem renunța la unele care sunt puternic corelate între ele.

CERINȚA MINIMĂ - analiza între un atribut numeric și sarcina de clasificare, un atribut categorial și sarcina de clasificare, un atribut numeric și sarcina de regresie, un atribut categorial și sarcina de regresie

3.2. Preprocesarea datelor [2p]

Folosind înțelegerea asupra datelor câștigată la pasul 3.1, putem aplica o serie de pași de preprocesare a datelor care să abordeze următoarea serie de posibile probleme.

1. Date lipsă pentru un atribut într-un eșantion

Pentru atribute cu valori lipsă se utilizează o **procedură de imputare**. Acestea pot fi univariate (să implice doar atributul pentru care lipsește o valoare) sau multivariate (valoarea atributului lipsă se obține în funcție de valorile celorlalte atribute din eșantion).
Exemple de imputare univariată sunt: valoarea medie, valoarea mediană, valoarea cea mai frecventă (en. mode). Imputările multivariate implică, de regulă, diverse forme de regresie peste valorile celorlalte atribute.

Sarcina este de a determina care atribute au date lipsă și să folosiți o metodă de imputare potrivită pentru acel atribut (a se vedea indicațiile din Anexă: Preprocesare Datelor)

CERINȚA MINIMĂ - imputarea într-o coloană însoțită de justificarea alegerii făcute

2. Valori extreme pentru un atribut într-un eșantion

Atributele numerice pot avea valori extreme (e.g. fie prea mici, fie prea mari în raport cu media sau mediana valorilor pentru acel atribut). Valorile extreme pun probleme în cazul unei clasificări, astfel că dorim să le identificăm în avans și să înlocuim valoarea lor printr-o procedură de imputare (le tratăm ca pe o valoare lipsă)

Sarcina este de:

- A determina dacă o valoarea a unui atribut numeric este o valoare extremă (a se vedea indicațiile din Anexă: Preprocesare Datelor)
- A înlocui acea valoare printr-una obținută prin imputare

CERINȚA MINIMĂ - identificarea outlierelor dintr-o coloană și tratarea lor

3. (OPȚIONAL) Atribute redundante (puternic corelate)

Dacă analiza de la punctul 3.1 indică prezența unor atribute numerice sau categorice puternic corelate, se va proceda prin eliminarea acelor atribute din setul celor folosite în predicție pentru a simplifica calculul algoritmului folosit.

!Atenție: Nu ne dorim să eliminăm atribute corelate cu obiectivele noastre. Există situații de overfitting în care apare o relație de corelare între un atribut și coloana obiectiv, care nu generalizează pe setul de validare, moment în care putem elimina acel atribut. Însă, în general ne dorim atribute sau grupări de atribute care să fie corelate cu sarcina, pentru a prezice pe baza unor trenduri din date.

Sarcina este de a argumenta în raport dacă ați eliminat vreun atribut pe seama analizei de redundanță a acestuia.

4. Plaje valorice de mărimi diferite pentru atributele numerice

Atributele numerice din setul de date pot avea scale valorice semnificativ diferite (e.g. valori de ordinul miilor pe un atribut, și de ordinul unităților pe un altul). În acest caz, algoritmi precum Regresia Logistică vor fi puternic afectați pentru că o combinație liniară a valorilor de atribute va fi dominată de atributul cu valori numerice foarte mari). Ca atare, dorim să standardizăm valorile atributelor numerice.

Sarcina este de a efectua operația de **standardizare** a valorilor atributelor numerice (a se vedea indicațiile din Anexă: Preprocesare Datelor)

3.3 Utilizarea algoritmilor de Învățare Automată [6p]

Metodologia cercetării - Ablația

În dezvoltarea unui model de inteligență artificială, există o anumită etapizare a modului în care se fac experimente, valabil în orice fel de sarcină de învățare. Să luăm exemplul arborilor de decizie, cunoscuți de la laborator:

- Putem să folosim biblioteca scikit-learn: Decision Tree Classifier
- Vom stabili o configurație pe care o cunoaștem: Max_depth = 10, algoritm de selecție = ID3 și de acolo vom citi despre anumiți hiperparametri în codul din scikit learn, astfel de biblioteci fiind riguros documentate.
- Vom observa că înțelegem semnificația hiperparametrului min_samples_leaf și atunci vom putea încerca o serie de experimente pe setul de validare:

Clasificare DT - ID3 - Variație min_samples_leaf

Min_samples_leaf	Precizie (↑)	Recall (↑)	F1 (↑)
2	0.4	0.4	0.2
5	**0.8**	**0.7**	**0.7**
10	0.6	0.9	0.65

Tab. [numărul_lui curent] - Influența modificării min_samples_leaf în evaluarea cantitativă

- Avem un tabel în care headerul ne prezintă tipul de metrici urmărite: (↑) - arată că metrica este mai bună cu cât este mai mare
- Într-un astfel de tabel ne concentrăm pe maxim 3 aspecte variate pentru a nu deveni greu de urmărit
- Când facem un tabel cu multe linii, pentru a ne scoate în evidență cele mai bune configurații, uzual punem bold pentru cel considerat cel mai bun, și subliniere pentru al doilea cel mai bun. Dar această legendă trebuie explicată și în comentariu.

- Dacă **NU ÎNȚELEGEM** semnificația unui hiperparametru după mai multe citiri, îl lăsăm cu valorile lui sugerate default și ne concentrăm pe aspectele pe care ne putem forma o intuiție și care pot ajuta spre îmbunătățirea performanței.
- După ce am stabilit min_samples_leaf, observ că nu este neapărat o corelație liniară între adăugarea mai multor modificări asupra infrastructurii modelului. Jurnalizarea treptată asupra mai multe aspecte, constructiv, cu și fără diverse componente pentru a justifica relevanța lor poartă numele de **Ablație** și arată astfel:

Algoritm de split	Min_samples_leaf	Modificări dataset	Precizie (↑)	Acurateță (↑)
ID3	3	-	0.4	0.2

	5	-	0.5	0.4
	3	+imputation	0.6	0.6
	5	+imputation	0.5	0.45
	5	+ outlier removal	0.7	0.7

Tab. [curent] + 1 - Contribuția împărțirii setului de date corelată cu min_smaples_leaf pentru ID3

- Primul rând se numește “baseline” și este modelul la care ne raportăm atunci când analizăm îmbunătățirile aduse prin rafinarea antrenării
- În situația ipotetică prezentată, putem observa cum modelul care stabilește etichete mai repede (min_sample_leaf mai mic) obține rezultate mai bune odată cu imputarea setului de antrenare, în timp ce pentru o valoare mai mare, nu se observă un câștig semnificativ pentru această procedură.
- Din acest punct, am putea izola exemple specifice (poartă numele de **ANALIZA CALITATIVĂ**) pentru a vedea ce determină această discrepanță.

- **AȘA NU -** Tentația persoanelor la început este să facă un GridSearch cu foarte mulți parametrii în produs cartezian și să rămână cu cea mai bună combinație de acolo: se pun toți algoritmii de selecție a nodurilor posibili (ID3, C4.5, C5.0, CART), cu 10 adâncimi posibile foarte apropiate, cu 5 min_impurity_decrease puse aleator - acest lucru e cronofag și nu contribuie la formarea unei intuiții despre logica în care au dat bine acei parametrii, deci NU este un setup dezirabil!
  - O astfel de abordare are sens în momentul în care încercăm să demonstrăm o îmbunătățire fundamentală - De exemplu, dacă am făcut un cod low-level care face un speed-up semnificativ asupra oricărui arbore de decizie, ne-ar interesa o astfel de analiză exhaustivă care să valideze îmbunătățirea în condiții diverse.

Clasificare [2p]

Ne dorim prin această temă, să ne concentrăm pe partea de funcționalitate, pe îmbunătățirea rezultatelor și înțelegerea hiperparametrilor modelelor folosite. Prin urmare, cerința **MINIMĂ** este folosirea unui **arbore de decizie** pentru stabilirea unui baseline de clasificare. Acest arbore poate fi implementat de voi manual, pe baza laboratorului sau poate fi importat din biblioteca sklearn - Decision Tree Classifier. Încercați minim un experiment în care să variați un hiperparametru al arborelui de decizie.
Ulterior, nu aveți nicio condiționare în ceea ce privește implementarea clasificatorilor pe care îi folosiți atât timp cât **ÎI ÎNȚELEGEȚI** și cât **EXPLICAȚI** nevoia utilizării lor.
Înțelegerea modelelor folosite va fi validată în momentul prezentării de către asistentul de laborator.
Recomandarea este să vă rezumați la algoritmii de clasificare discutați la curs, deoarece veți avea ocazia la materiile următoare să antrenați modele neuronale adânci.

Înaintea rulării propriu-zise a algoritmului țineți cont de următoarele:

- Dacă folosim biblioteci, anumiți algoritmi produc erori în momentul în care primesc inputuri categoriale. Pentru a putea folosi acești algoritmi, variabilele **categoriale** (incluzând-o pe cea țintă) trebuie convertite într-o formă numerică. Utilizăm în acest scop

clase disponibile în biblioteca scikit-learn: LabelEncoder sau OneHotEncoder. De exemplu, putem folosi LabelEncoder pentru a encoda variabila țintă și OneHotEncoder pentru a encoda variabilele predictor de tip categoric.
Un exemplu de encodare în stil one-hot există și în laboratorul de Arbori de Decizie, unde s-a utilizat metoda pandas.get_dummies.

Punctarea se face astfel - **vom considera rezultatele obținute de voi pe setul de testare lăsat public în cerința temei**:

1. **Aplicarea soluției dummy pe setul vizibil de testare pe care vi l-am dat - model underfit [0.5p]**
    - Dacă soluția voastră este doar un apel de sklearn, fără antrenare, fără niciun experiment cu privire la hiperparametri.
2. **Depășirea baseline-ului din date [1p]**
    - Dacă soluția voastră depășește o acuratețe de 0.25
3. **Perfecționarea [1-2p]**
    - Maxim dacă una dintre soluțiile voastre depășește o acuratețe de 0.6
    - Vom scala liniar rezultatul vostru maxim între 0.25 și 0.6.

Documentați în raportul vostru abordarea folosită pentru clasificare:

- Tipul de encodare folosit pentru fiecare atribut categoric
- Modelele pe care ați decis să le utilizați, împreună cu seturile de hiperparametri și experimentele pe care ați decis să le încărcați
- Jurnalizarea pașilor spre cea mai bună soluție a voastră

Regresie [2p]

Respectând același deziderat precum la sarcina de clasificare, cerința **MINIMĂ** este folosirea unei regresii liniare pentru stabilirea unui baseline de regresie. Acest model poate fi implementat de voi manual, pe baza laboratorului sau poate fi importat din biblioteca sklearn - LinearRegression. Încercați minim un experiment în care să adăugați și variați factorul de regularizare.
Ulterior, nu aveți nicio condiționare în ceea ce privește implementarea regresorilor pe care îi folosiți atât timp cât **ÎI ÎNȚELEGEȚI** și cât **EXPLICAȚI** nevoia utilizării lor.
Pentru acest task va fi nevoie să creați grafice cu performanța la antrenare și validare a modelelor de regresie:

- Pentru a compara comportamentul la antrenament al mai multor modele, vom suprapune graficele de antrenament:

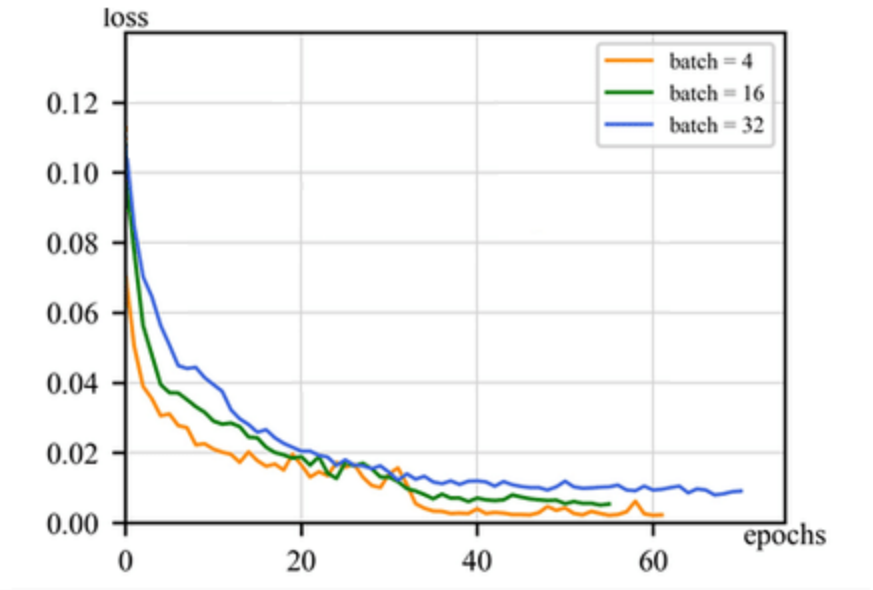


Fig [nr_figură]. Evoluția comparativă pe setul de antrenare a mai multor experimente

- Pentru a observa dacă modelul nostru face overfit, suprapunem curbele de train și test / validation pe același grafic.

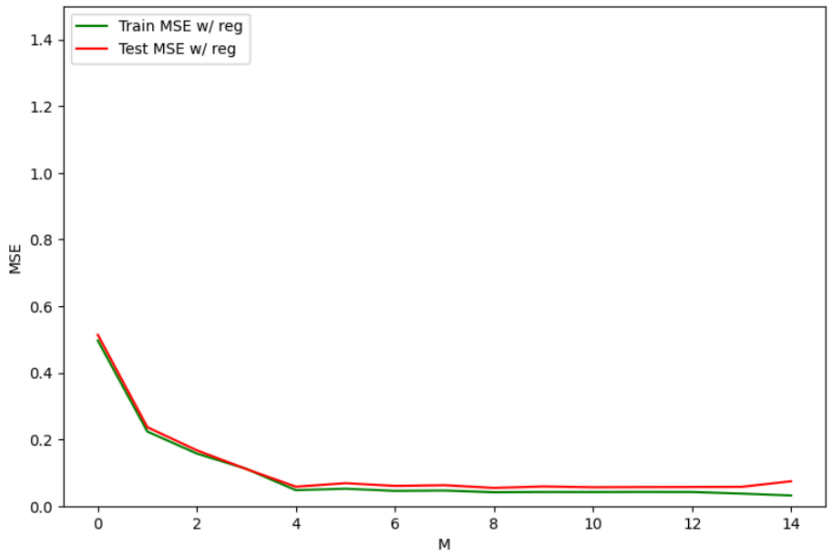


Fig [nr_figură] + 1. Comportament comparativ pe setul de antrenare versus cel de validare pentru același experiment

- **AȘA NU -** Nu punem niciodată mai mult de 10 curbe pe același grafic, deoarece devine ilizibil. Nu punem performanțe de train și validation în același timp pe mai multe experimente în aceeași imagine, preferăm să divizăm plotul în 2 grafice, unul cu comparația pe train și unul cu comparația pe test. Puteți observa în figura de mai jos că nu puteți urmări prea multe din cauza încărcării graficului.

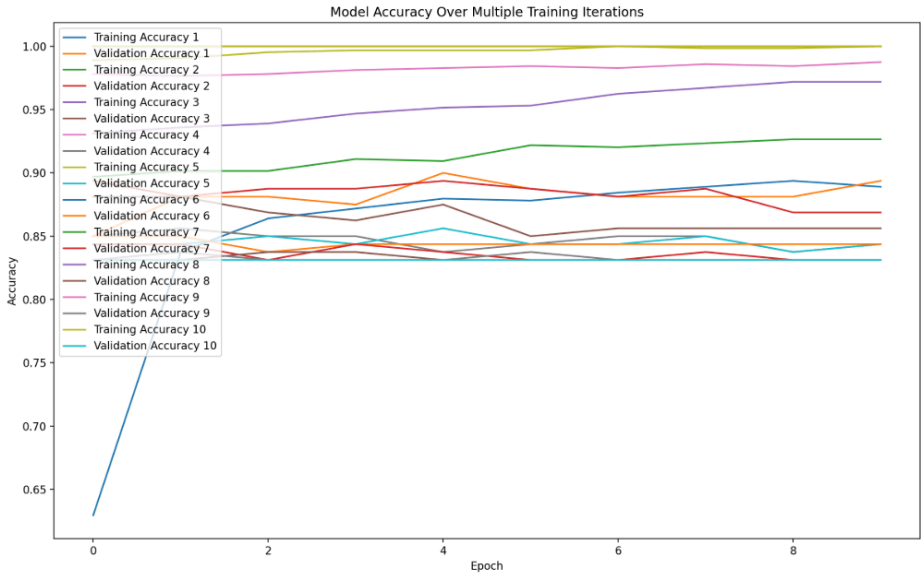


Fig [nr_figură] + 2. Exemplu cu prea multe grafice de antrenare și validare

Punctarea se face astfel - **vom considera rezultatele obținute de voi pe setul de testare lăsat public în cerința temei**:

4. **Aplicarea soluției dummy pe setul vizibil de testare pe care vi l-am dat - model underfit [0.5p]**
    - Dacă soluția voastră este doar un apel de sklearn, fără antrenare, fără niciun experiment cu privire la hiperparametri.
5. **Depășirea baseline-ului din date [1p]**
    - Dacă soluția voastră depășește MAE/MSE/RMSE de (CA: 266/13/16)
6. **Perfecționarea [1-2p]**
    - Maxim dacă una dintre soluțiile voastre este mai bună decât MAE/MSE de (CA: 48/5/7)
    - Vom scala ponderat rezultatul vostru maxim între pragurile din baseline și perfecționare.

Documentați în raportul vostru abordarea folosită pentru regresie:

- Descoperirile proprii cu privire la tipul de regularizare
- Modelele pe care ați decis să le utilizați, împreună cu seturile de hiperparametri și experimentele pe care ați decis să le încărcați
- Jurnalizarea pașilor spre cea mai bună soluție a voastră

Evaluarea algoritmilor [2p]

CERINȚA MINIMĂ - În raportul vostru trebuie să prezentați **cel puțin** următoarele:

- Descrieți **setul de hiperparametri** final pe care i-ați folosit **pentru fiecare algoritm** în parte.

- Pentru **fiecare algoritm folosit la clasificare**, realizați o **matrice de confuzie** peste clase pentru **cel mai bun model**, pentru a observa care sunt clasele pentru care algoritmul greșește în clasificare

- Realizați **MINIM** un tabel comparativ al algoritmilor precum cele din exemple. În tabel prezentați ***acuratețea generală de clasificare, precizie / recall / F1*** **la nivelul fiecărei clase în parte** (definiții ale metricilor pe Wikipedia)
    - Pe linii va fi indexată numele algoritmului
    - Pe coloane vor fi prezentate metricile cerute
    - **Relevați prin bolduire** valorile maxime pentru fiecare metrică
    - Similar cu exemplele din **secțiunea Metodologia cercetării**

- Pentru **sarcina de regresie** furnizați graficele **curbelor de eroare** pentru seturile de date de antrenare și validare. Trasați curbele în concordanță cu explicațiile oferite în secțiunea sarcinii de Regresie.

- Realizați **MINIM** un tabel comparativ al algoritmilor precum cele din exemple. În tabel prezentați minim o măsurătoare cantitativă ***MAE / MSE / RMS / R^2.***

- **Comentați asupra rezultatelor, explicând de ce credeți că algoritmul cu rezultatul cel mai bun obține această performanță.**

4. Predarea temei

Tema se trimite ca o arhivă .zip care conține cel puțin 2 fișiere:

- un **fișier Python** sau **Jupyter Notebook** care contine implementarea analizelor cerute
- un **fișier PDF** care prezintă **raportul de rezultate și interpretare a** analizelor cerute
    - **Cerința 3.1** - cuprinde toate vizualizările și statisticile cerute. **Este obligatorie** prezența în text a **unei interpretări personale / analize** a diagramelor sau tabelelor rezultante. (**Exemplu**: “Pe baza faptului că Regresia Liniară este foarte puternic afectată de existența outlierelor, pe baza unei analize a variației din coloana X, am decis eliminarea ei din tratarea sarcinii de clasificare.”

    - **Cerința 3.3** - include **raportarea preprocesării datelor** și a evaluării algoritmilor de clasificare și regresie. **Este obligatorie** prezența în text a **unei interpretări / analize** a rezultatelor obținute (e.g. care este impactul dezechilibrului de clase - dacă el există - asupra performanțelor, cât de puternic este impactul hiperparametrilor asupra performanței fiecărui algoritm considerat, care sunt clasele cu cele mai bune predicții, pe ce se comportă cel mai bine regresia și cu ce fel de regularizare și de ce).

! Atenție - evaluarea punctajului din leaderboard va fi condiționată de existența unei jurnalizări a experimentelor dezvoltate până la cea mai bună performanță.
! Folosirea seed hacking duce la penalități de până la 3p din valoarea temei - scopul este prezentarea străduinței, a realizării etapizate a îmbunătățirilor, nu de obținere cu orice preț a celor mai mari metrici posibile.
! Scopul se pune asupra înțelegerii comportamentelor algoritmilor prezentați, încercați pe cât posibil să nu faceți experimente de tip GridSearch cu foarte multe configurații aleatorii - vezi secțiunea Metodologia cercetării.

5. Încărcarea pe platformă - BONUS [2p]

1p - clasificare, 1p - regresie
Din moment ce am întârziat cu pregătirea platformei, orice submisie pe platformă va fi considerată bonus și va fi scalată liniar pe baza metricii de pe platformă.

Submisiile sunt evaluate în funcție de Acuratețe pentru Clasificare și MSE pentru Regresie între clasa prezisă și țintă. **Dacă**
Pentru fiecare ID din setul de testare de pe platforma Kaggle și predicția sa corespunzătoare obținută de voi pentru taskul curent. Fișierul trebuie să conțină un antet și să aibă următorul format:

id, prediction
1,[valoare clasificare/regresie]
2,[valoare clasificare/regresie]

CA:
Clasificare: https://www.kaggle.com/t/f4096b2dc5344e39b6c8ab0f0b6445af
Regresie: https://www.kaggle.com/t/014f4a24fdfb4b67bd52178c5689a8ca

! Punctarea pentru bonus incepe o data cu depășirea benchmarkurilor de pe platforma kaggle (pentru clasificare - o predictie cu aceeași clasă pentru toate intrările, pentru regresie, o predicție cu 0), si se scaleaza proporțional până la maxim pentru metricile din sectiunea "Perfecționare" sugerate pentru validarea locală.

Alternativ, temele ce respectă o structură coerentă, prezintă un șir logic al experimentelor, au un mai multe experimente pentru fiecare sarcină sunt eligibile pentru un bonus de maxim 2p.!

6. Anexă

În rezolvarea temei este încurajată utilizarea a două biblioteci pentru sarcinile de analiză, preprocesare și aplicare a algoritmilor de predicție: **pandas** și **scikit-learn**, ambele în python.

Biblioteca Pandas este foarte frecvent utilizată pentru citirea, procesarea și analiza datelor livrate în format tabelar, așa cum este cazul seturilor de date din tema.
Revizitați indicațiile din laboratorul despre Arbori de Decizie și Păduri Aleatoare pentru a vedea exemple tipice de lucru cu date în pandas, îndeosebi, lucrul cu pandas.DataFrame.

Scikit-learn este o bibliotecă ce conține implementări a multor proceduri de prelucrare statistică a datelor, precum și a algoritmilor clasici de ML care au fost studiați și în cadrul laboratorului (e.g. Decision Trees, Random Forest, Logistic Regression, Multi-Layered Perceptron).

O altă bibliotecă renumită în competițiile de inteligență artificială pentru viteza de antrenare a modelelor, mai ales arbori de decizie și de regresie, și pentru rapiditatea convergenței spre performanțe foarte bune pe date tabulare este CATBoost. Vă încurajăm să instalați dependețele sale și să o utlizați, dar considerăm că ar fi benefic să vă familiarizați la începutul temei cu Scikit-learn întâi!

6.1. Explorarea datelor

Citirea datelor

Citirea datelor din fișiere CSV se poate face cu pandas.read_csv ceea ce va rezulta în obținerea unui DataFrame.
Atenție: datele lipsă din seturile de date furnizate sunt trecute ca spațiu gol în CSV-uri. Pandas va interpreta valoarea goală drept **numpy.NaN**, iar existența unei valori de tip NaN poate fi verificată prin pandas.isna.

Analiza atributelor numerice

Pentru a determina statistici generale despre coloane cu atribute numerice se poate utiliza apelul **pandas.DataFrame.describe.**
Afișarea statisticilor în grafice de tipul **boxplot** se poate face direct dintr-un DataFrame folosind metoda pandas.DataFrame.boxplot.

Analiza atributelor categorice sau ordinale

Pentru a determina valorile unice ale unei coloane de atribute categorice dintr-un DataFrame se poate utiliza metoda pandas.unique.
Realizarea unei histograme cu frecvența de apariție a fiecărei valori de atribut se poate face direct dintr-un DataFrame prin metoda pandas.Series.hist (HINT: Fixați și parametrul width pentru histogramă pentru afișare mai plăcută).

Pentru obținerea unei serii Pandas indexată după valorile unui atribut și numărând exemplele care au acea valoare poate fi folosită metoda pandas.DataFrame.value_counts.

De notat: Aceste metode pot fi folosite și pentru **analiza dezechilibrului de clase**.

Analiza corelației între atribute

Analiza corelației între **atribute numerice** se poate face direct dintr-un DataFrame prin metoda pandas.DataFrame.corr, utilizând criteriul Pearson.
O interpretare vizuală a gradului de corelare se poate obține cu metoda **matshow()** din **matplotlib,** ca în exemplul de aici.

Pentru atribute categorice, corelația între două atribute se poate evalua prin testul statistic Chi-Pătrat, a cărui ipoteză nulă este că variabilele sunt independente (nu sunt corelate), iar testul poate confirma sau infirma această ipoteză.
Un exemplu de evaluare se poate vedea aici, unde sunt utilizate metode din două biblioteci (pandas și scipy).

6.2. Preprocesarea datelor

Determinarea valorilor extreme

Determinarea valorilor extreme pentru atribute numerice se face pe baza statisticilor de bază ale acestora obținute prin metode precum pandas.describe().
În particular, metoda ce utilizează **diferența inter-quartilă (inter-quartile range - IQR)** poate fi folosită în cazul seturilor de date furnizate.

Un exemplu de utilizare se găsește aici. De notat că se poate încerca setarea quantilelor și la valori mai extreme, precum Q1 = 0.1 și Q3 = 0.9, în cazul în care variabilele au o plajă foarte largă de valori, pentru a distinge valori outlier, de simple valori mari.

Imputarea valorilor lipsă sau extreme

În procedura de imputare se pot utiliza două clase tipice din biblioteca **scikit-learn:** SimpleImputer (imputare univariată) și IterativeImputer (imputare prin raport la celelalte atribute din eșantion).

Exemple de utilizare a acestora se pot observa în documentația de aici.

De notat că puteți încerca aplicarea mai multor metode de imputare pe un anumit set de date, obținând astfel diferite transformări ale acestuia. Algoritmii de predicție pot fi rulați apoi pe fiecare transformare în parte, păstrând metoda de imputare ce oferă cele mai bune rezultate.

Standardizarea datelor

Pentru procedura de standardizare a datelor se poate recurge la clasele tipice existente în **scikit-learn.** Exemple de utilizare se găsesc aici.
Acordați atenție îndeosebi claselor **StandardScaler**, **MinMaxScaler** sau **RobustScaler.**